# Yanasse: Finding New Proofs from Deep Vision's Analogies

## Part 3: Probability Theory → Condensed Mathematics

Alexandre Linhares*

April 2026

**Abstract**

This is Part 3 of the Yanasse series, applying cross-area tactic transfer to the pair Probability → Condensed. Condensed mathematics—the sheaf-theoretic framework of Clausen and Scholze that replaces point-set topology—is structurally the most distant target tested so far: it was designed to *eliminate* the filter-based and $\sigma$-algebra-based vocabulary on which probability-theory tactics depend.

The method produces 2 Lean-verified new proofs out of 10 transfer attempts (20%), the lowest success rate in the series. Both proofs target the same theorem—the triangle identity of the adjunction between condensed sets and topological spaces (`topCatAdjunction.left_triangle_components`)—and both transfer the "go pointwise" pattern that probability theory uses to reduce equalities of structured objects (kernels, random variables) to per-element obligations.

The central finding is a *strictly better proof*: the transferred tactic `congr with x` replaces `ext` *without* requiring the `set_option backward.isDefEq.respectTransparency false` workaround that the existing Mathlib proof needs. This is not a paraphrase of the original proof but a genuine improvement enabled by cross-area transfer. The remaining 8 schemas all fail for a common root cause: condensed mathematics systematically avoids the type-theoretic structures (filters, $\sigma$-algebras, conditional definitions, `CanLift` instances) that probability-theory tactics operate on.

Cross-pair comparison across all three target areas confirms a sharp dichotomy: schemas that interact with measure-theoretic *types* never transfer; schemas that operate on *proof architecture* (decomposition, case-split, congruence) transfer when the target has compatible goal shapes.

*alexandre@linhares.ltd. Affiliations: Getulio Vargas Foundation and ARGO LABS

# Contents

**1 Introduction** **3**
1.1 Overview of the Deep Vision framework . . . . . 3
1.2 Results preview . . . . . 3

**2 Methodology** **4**
2.1 Pipeline summary . . . . . 4
2.2 Pair-specific parameters . . . . . 4

**3 The Target: Condensed Mathematics** **5**

**4 Results: Probability → Condensed** **5**
4.1 New proof P1: `congr with x`replaces `ext`without transparency workaround . . . . . 6
4.2 New proof P2: `ext1 x`with named variable . . . . . 7

**5 Cross-Pair Comparison: Three Target Areas** **8**

**6 The (Head, Modifier) Decomposition: Three-Area Evidence** **9**

**7 Failures and Their Diagnoses** **9**

**8 Discussion** **11**
8.1 The 20% success rate . . . . . 11
8.2 Convergence to a single target theorem . . . . . 11
8.3 The `set_option`finding . . . . . 11
8.4 Cost . . . . . 11

**9 Cumulative Results After Three Pairs** **11**

**10 Conclusion** **12**

**A Lean Code: New Proofs** **15**
A.1 Item 21: `left_triangle_components`via `congr with x`(no `set_option`) . . . . . 15
A.2 Item 22: `left_triangle_components`via `ext1 x`(three variants) . . . . . 16



*Citation: Linhares, A. "Yanasse: Finding New Proofs from Deep Vision's Analogies — Part 3," Argolab.org Report for Dissemination, 2026.*

# 1 Introduction

Parts 1 and 2 of this series [1, 2] demonstrated cross-area tactic transfer at a 40% schema success rate on Probability → Representation Theory and Probability → Set Theory alike. Both target areas, while mathematically distant from probability, share a common property: they use the standard vocabulary of Lean's type theory—equalities, inequalities, type-class instances—in ways that probability-theory proof patterns can engage with.

This paper tests the method against a fundamentally different target. Condensed mathematics, developed by Clausen and Scholze [16, 17], replaces point-set topology with a sheaf-theoretic framework: a condensed set is a sheaf on the category of compact Hausdorff spaces. The theory was designed precisely to avoid the categorical deficiencies of topological spaces (the category **Top** lacks exactness properties), and in doing so it eliminates the filter-based and measure-theoretic vocabulary that probability-theory tactics depend on. This makes CONDENSED the hardest target in the Yanasse series: every domain-gated schema faces not just an unfamiliar target but an *actively hostile* one.

The Lean formalization of condensed mathematics in Mathlib [19, 20] is comparatively young: 189 extracted proof states across 33 files, versus 672 for Representation Theory and 2,076 for Set Theory. This small pool increases the risk of QAP score saturation (multiple targets tied at the same score) and limits the diversity of available target theorems.

## 1.1 Overview of the Deep Vision framework

The Deep Vision framework [3,13], growing out of the Fluid Analogies Research Group tradition [4–6], represents any structured domain as a *relational network*: entities connected by typed, directed relations. An analogy optimization finds the entity permutation that maximizes relational consistency between two networks. The matching code (`deep_vision_lib.py`) is entirely domain-independent: it receives two relational structures and returns a similarity score and entity correspondence, without knowing which domain it is processing.

**Chess analogies.** In chess [7–12], entities are pieces and relations are attack, defense, blocking, confinement, and pinning. Two pieces in entirely different positions can play "the same role" if their relational profiles match.

**From chess to proofs.** The same matching engine, with only the relation extractor swapped, handles Lean 4 proof states: entities become hypotheses, goals, and types; relations become rewrite, fit/apply, head-match, structure, equality, reflexive, witness, bidirectional, kernel/simp, decidable, and lemma-needed (14 relation types). The proof-state matcher has previously been used to discover a formal proof of Wolstenholme's theorem [14] in Lean 4 (see also [15]).

## 1.2 Results preview

Applying the method to Probability → Condensed produces:

- 2 Lean-verified new proofs from 2 successful schema transfers out of 10 attempts (20% transfer success rate).
- A *strictly better proof*: the transferred `congr with x` avoids a transparency workaround (`set_option`) that the existing Mathlib proof requires (Section 4.1).

- A paradigm-incompatibility diagnosis: 8 of 10 schemas fail because condensed mathematics systematically eliminates the type-theoretic structures they require (Section 7).
- Three-way cross-pair comparison across all target areas tested so far (Section 5).

## 2 Methodology

The methodology is identical to Parts 1 and 2 [1, 2]. We summarize the pipeline and note the parameters specific to this pair.

### 2.1 Pipeline summary

1. **Schema extraction.** Parse the Mathlib proof-state corpus (217,133 entries). Each tactic invocation becomes a schema tuple (*head*, *arity*, *has_with*, *uses_lemma*).
2. **$z$-score ranking.** For each (area, schema) pair, compute the $z$-score of the schema's frequency across 27 areas. A schema with $z \geq +2$ in the source and $z \leq -1$ (or absent) in the target is a transfer candidate.
3. **GPU-accelerated analogy matching.** Sample 8 source proofs per schema, match each against all 189 Condensed proof states using the `BatchMatcher` on MPS. Retain the top-5 target analogues by normalized score.
4. **Semantic adaptation.** A fresh AI reasoning session (Claude Opus, 50-minute budget) adapts the source tactic's *mathematical strategy*—not its syntax—to the target theorem.
5. **Lean verification.** The adapted proof is compiled with `lake env lean`. Exit code 0 with no `sorry` constitutes verification.

### 2.2 Pair-specific parameters

- **Source:** `Mathlib.Probability` (30,384 tactic invocations, the largest Mathlib area).
- **Target:** `Mathlib.Condensed` (189 proof states—the smallest target pool in the series, $\sim$9$\times$ smaller than Set Theory and $\sim$3.5$\times$ smaller than Representation Theory).
- **Matching cost:** $8 \times 189 = 1{,}512$ individual analogy computations per schema, completing in approximately 2–10 minutes on MPS—significantly faster than the previous pairs due to the smaller pool.
- **Top-10 transfer candidates** (by gap score): `filter_upwards`, `congr with`, `fun_prop` (as `all_goals fun_prop`), `ext1`, `lift`, `any_goals`, `measurability`, `by_cases`, `congrm`, `split_ifs`.

Note the substitutions relative to previous pairs: `all_goals fun_prop` replaces bare `fun_prop` (the full combinator, not just the inner tactic, is the actual source pattern), and `split_ifs` replaces `push` (the next-highest-gap schema for this target).

## 3 The Target: Condensed Mathematics

A *condensed set* is a sheaf of sets on the category **CompHaus** of compact Hausdorff spaces equipped with the coherent topology [16]. The functor

$$T\colon \mathbf{Top} \longrightarrow \mathbf{CondSet}, \qquad X \longmapsto \big(S \mapsto \mathrm{Cont}(S, X)\big)$$

embeds topological spaces into condensed sets, and admits a left adjoint

$$U\colon \mathbf{CondSet} \longrightarrow \mathbf{Top}$$

that recovers the "underlying topological space" of a condensed set. The adjunction $U \dashv T$ is the categorical backbone of the Mathlib formalization [19]: its unit $\eta\colon \mathrm{Id} \Rightarrow T \circ U$ and counit $\varepsilon\colon U \circ T \Rightarrow \mathrm{Id}$ satisfy the triangle identities

$$\varepsilon_{T(X)} \circ T(\eta_X) = \mathrm{id}_{T(X)}, \qquad U(\varepsilon_Y) \circ \eta_{U(Y)} = \mathrm{id}_{U(Y)}. \tag{1}$$

Both new proofs in this paper target the second identity (Equation 1, right), formalized in Mathlib as `topCatAdjunction.left_triangle_components`.

**Why condensed math is hard to reach from probability.** The Clausen–Scholze framework was designed to replace filter-based topology—the very vocabulary that probability theory's most characteristic tactics (`filter_upwards`, `measurability`, `fun_prop`) operate on—with sheaf-theoretic conditions. Condensed proofs reason about *functors*, *natural transformations*, and *adjunctions*, not about $\sigma$-algebras, filters, or conditional definitions. This makes the pair a stress test for cross-area transfer: any success must come from proof patterns that transcend the measure-theoretic vocabulary.

The Mathlib formalization draws on the work of Asgeirsson [19, 20], building on the foundations laid by the Liquid Tensor Experiment [21, 22], which formalized Scholze's theorem on liquid vector spaces in Lean and demonstrated that condensed mathematics could be made fully rigorous in a proof assistant.

## 4 Results: Probability → Condensed

Table 1: Transfer results for Probability → Condensed.

| Item | Schema head | `with` | Ar. | Result | Adaptation |
|---|---|---|---|---|---|
| 20 | `filter_upwards` | ✓ | 1 | Failed | Paradigm gap (no Filter API) |
| 21 | `congr` | ✓ | 0 | **New proof** | `congr with x` (no `set_option`) |
| 22 | `ext1` | | 1 | **New proof** | `ext1 x` (named variable) |
| 23 | `all_goals` | | 1 | Failed | No multi-goal sites |
| 24 | `lift` | ✓ | 5 | Failed | No `CanLift` instances |
| 25 | `any_goals` | | 1 | Failed | Single sequential goal |
| 26 | `measurability` | | 0 | Failed | No $\sigma$-algebra predicates |
| 27 | `by_cases` | | 4 | Failed | No conditional definitions |
| 28 | `congrm` | | 5 | Failed | No shared outer operators |
| 29 | `split_ifs` | | 0 | Failed | Zero `ite`/`dite` |

Two transfer attempts succeeded, both targeting the same theorem: the left triangle identity of the condensed-set/topological-space adjunction (`topCatAdjunction.left_triangle_components`).

### 4.1 New proof P1: `congr with x` replaces `ext` without transparency workaround

**Theorem.** For a condensed set $Y$, the triangle identity

$$U(\varepsilon_Y) \circ \eta_{U(Y)} = \mathrm{id}_{U(Y)}$$

holds in **Top**, where $U$ = `condensedSetToTopCat`, $\eta$ = `topCatAdjunctionUnit`, and $\varepsilon$ = `topCatAdjunctionCounit`.

**Context.** The triangle identities are the defining equations of an adjunction [18]. In the condensed setting, they encode the fact that the "condensify then topologize" round-trip is coherent: applying the unit and then the counit (or vice versa) recovers the identity. The Mathlib formalization follows Asgeirsson [19, 20], who developed the condensed-set infrastructure building on the Liquid Tensor Experiment [21, 22]. The specific theorem `left_triangle_components` appears in `Mathlib.Condensed.TopCatAdjunction`, one of the core files establishing the $U \dashv T$ adjunction.

**The existing Mathlib proof.** The current proof requires a global transparency override:

```
set_option backward.isDefEq.respectTransparency false in
theorem left_triangle_components :
    condensedSetToTopCat.map (topCatAdjunctionUnit Y) ≫
      topCatAdjunctionCounit Y.toTopCat = ⊮ _ := by
  ext
  change Y.obj.map (⊮ _) x = x
  simp
```

Without `set_option backward.isDefEq.respectTransparency false`, the `ext`-based proof fails: `ext` applies the registered `@[ext]` lemma for `TopCat.Hom`, which leaves the goal in a form that interacts poorly with Lean's backward definitional-equality transparency settings.

**New proof (congr with x).** The transferred tactic replaces `ext` with `congr with x`, which operates via congruence closure rather than extensionality lemma application:

```
-- No set_option needed
example (Y : CondensedSet.{u}) :
    condensedSetToTopCat.map (topCatAdjunctionUnit Y) ≫
      topCatAdjunctionCounit Y.toTopCat = ⊮ _ := by
  congr with x
  change Y.obj.map (⊮ _) x = x
  simp
```

**Why the transfer works.** `congr with x` performs two operations: `congr` recognizes that both sides of the morphism equality share the same outermost constructor (`ContinuousMap.mk`—a function paired with a continuity proof) and decomposes accordingly; `with x` introduces a point $x$ in the underlying topological space. The residual goal $Y$.`obj.map` $(\not\Vdash \_)\, x = x$ follows from the sheaf functoriality axiom via `simp`.

The key difference from `ext`: congruence closure pattern-matches on term structure directly, bypassing the extensionality-lemma mechanism that triggers the transparency issue. The two tactics achieve the same mathematical decomposition (morphism equality → pointwise equality) via different mechanisms, and the congruence route is immune to the backward-`isDefEq` interaction.

**Source pattern.** In `Mathlib.Probability.Gaussian.HasGaussianLaw.Independence`, the tactic `congr with i` decomposes an equality of characteristic-function products $\prod_i \varphi(X_i)(\cdot) = \prod_i \exp(\cdots)$ to per-factor equality on the index $i$. The pattern—"both sides share an outermost indexed operator; `congr` decomposes, `with` names the index"—is the probability-theory workhorse for equalities of kernels, expectations, and products.

**Cross-pair comparison.** This is the third target area for schema 138 (`congr with`):

| Target | Head transfers? | Modifier transfers? | Net result |
|---|---|---|---|
| RepTheory (Part 1) | No ($\leq$ goal) | Yes (`rintro`) | Adapted proof |
| SetTheory (Part 2) | No (Cardinal) | Yes (positive controls) | Failed (wrong rank-1) |
| **Condensed (Part 3)** | **Yes** (= on ContinuousMap) | **Yes** (`with x`) | **Strictly better proof** |

Condensed is the only target where both the head (`congr`) and the modifier (`with`) transfer directly. The reason: the target goal is an equality of morphisms in a concrete category (continuous maps between topological spaces), which has exactly the constructor-shaped structure that `congr` expects.

Copy-and-pasting the Lean code of the proofs above to Google's Gemini LLM with no additional context has yielded:

> That is an excellent refactor! Switching from `ext` to `congr` is a savvy move. While `ext` tries to apply every possible extensionality lemma in the database, `congr` (congruence) is more targeted—it basically says "these two sides are the same if their components are the same." The fact that you were able to remove the `set_option` suggests that `congr` navigated the "transparency" of the definitions more effectively than `ext` did. In Lean, `congr` is often better at "drilling through" categorical structures because it focuses on the function application rather than just the object type.
>
> **Why this version is "Clean."**
>
> **No Options:** Removing `set_option` makes the proof more portable and less likely to cause weird unification behavior in other parts of your file.
>
> **Targeted:** `congr with x` is slightly more explicit about what you are identifying than a generic `ext`.
>
> **Frugal:** This is a very "short" proof for a concept that is mathematically deep (the first triangle identity of an adjunction).

### 4.2 New proof P2: `ext1 x` with named variable

**Theorem.** Same as P1: `topCatAdjunction.left_triangle_components`.

**New proof (ext1 x, three variants).**

1. **ext1 x + change + simp.** Identical to the original proof but with `ext1 x` (named variable) instead of bare `ext`. Still requires `set_option`.
2. **ext1 x + change + simp only.** Fully explicit lemma list: `FunctorToTypes.map_id_apply`, `condensedSetToTopCat_obj_carrier`, etc.
3. **ext1 x + change + rw + rfl.** A rewrite chain mimicking the probability-theory pattern: `ext1 x` → `rw [FunctorToTypes.map_id_apply]` → `rfl`.

**Source pattern.** In probability, `ext1` $\omega$ reduces set equality to pointwise membership $\forall \omega,\ \omega \in S \leftrightarrow \omega \in T$; `ext1 a` reduces kernel equality to pointwise measure equality $\forall a,\ \kappa_1(a) = \kappa_2(a)$. The arity-1 form (naming the variable) is standard in probability because subsequent rewrite chains reference it.

**Adaptation.** `ext1 x` compiles directly because `TopCat` morphisms have a registered `@[ext]` lemma. The transfer is syntactic (named variable convention), not semantic. Unlike P1, this proof still requires the `set_option` workaround—`ext1` applies the same extensionality lemma as bare `ext` and triggers the same transparency interaction.

**Domain asymmetry.** In probability, after `ext1 x`, the goal is in *rewrite-normal form*: `simp` lemmas (`map_apply`, `prod_apply`) directly simplify both sides. In condensed math, after `ext1 x`, the goal passes through layered categorical abstractions that are *definitionally* but not *rewritably* equal to the desired form. The `change` tactic bridges this gap by asking Lean's kernel to verify the definitional equality.

**Relationship to P1.** P1 (`congr with x`) is strictly stronger: it achieves the same mathematical decomposition without the `set_option` workaround. P2 (`ext1 x`) demonstrates that the probability naming convention transfers, but the proof is not improved.

## 5 Cross-Pair Comparison: Three Target Areas

With three pairs of the Yanasse Project completed, we can compare schema behavior across three structurally different target areas. Seven schemas were tested in all three areas (Table 2).

Table 2: Schema transfer results across three target areas. ✓= new proof; × = failed.

| Schema head | RepTheory | SetTheory | Condensed | Pattern |
|---|---|---|---|---|
| `filter_upwards` | ✓ | × | × | Mixed |
| `congr with` | ✓ | × | ✓ | Mixed |
| `ext1` | × | ✓ | ✓ | Mixed |
| `lift` | × | × | × | Always fails |
| `measurability` | × | × | × | Always fails |
| `any_goals` | ✓ | ✓ | × | Mixed |
| `by_cases` | ✓ | ✓ | × | Mixed |
| `congrm` | × | × | × | Always fails |

**Finding 1** (The type-gated / architecture dichotomy)**.** Schemas partition into two groups:

- **Type-gated** (`lift`, `measurability`, `congrm`): require specific type-theoretic structures (CanLift instances, $\sigma$-algebra predicates, composite operator skeletons) that exist only in measure-theoretic areas. These fail in *every* non-measure-theoretic target—0/9 transfers across three areas.
- **Architecture-sensitive** (`filter_upwards`, `congr with`, `ext1`, `any_goals`, `by_cases`): operate on proof structure (decomposition, case-split, extensionality) and transfer when the target has compatible goal shapes. These succeed 8/15 times, but each schema's success depends on target-specific compatibility.


*Citation: Linhares, A. "Yanasse: Finding New Proofs from Deep Vision's Analogies — Part 3," Argolab.org Report for Dissemination, 2026.*

The type-gated group is now confirmed with three independent negative results per schema. Future Yanasse runs should skip `lift`, `measurability`, and `congrm` for any non-measure-theoretic target, saving ∼30% of compute.

**Why condensed is the hardest target.** Condensed mathematics achieves a 20% success rate versus 40% (RepTheory and SetTheory). The difference is not accidental: condensed proofs are *categorically sequential* (one goal at a time, handled by categorical lemma application), while both other targets have goals with algebraic or combinatorial structure amenable to case-splitting (`by_cases`), broadcasting (`any_goals`), or conditional unfolding.

- `by_cases` succeeds in RepTheory (morphism triviality split) and SetTheory (infinite/finite dichotomy) but fails in Condensed: no conditional definitions (`ite`/`dite`) exist to unfold, and no structural dichotomies create two-branch proof architectures.
- `any_goals` succeeds in RepTheory (heterogeneous sub-goals after decomposition) and SetTheory (arithmetic side-conditions) but fails in Condensed: proofs never have more than one open goal simultaneously.
- `filter_upwards` fails in both SetTheory and Condensed, but for different reasons: SetTheory has no `Filter` API (ordinal arithmetic is algebraic); Condensed was *designed* to replace filters with sheaf conditions.

## 6 The (Head, Modifier) Decomposition: Three-Area Evidence

The (head, modifier) decomposition introduced in Part 1 receives its strongest evidence from the three-area comparison of `congr with`:

**Finding 2** (Full schema transfer)**.** `congr with x` is the only schema to achieve full head + modifier transfer in any target. It does so only in Condensed—the area where the target goal happens to be an *equality of constructor-shaped terms in a concrete category*, matching exactly the goal structure that `congr` expects. In Representation Theory, only the modifier (`with`) transferred (the head required adaptation because the goal was $\leq$, not $=$). In Set Theory, both failed at rank-1 (Cardinal has no extensionality theorem), though a genuine transfer existed at rank-3.

This finding refines the (head, modifier) decomposition: whether the head transfers depends on *goal shape* (equality of structured terms), while whether the modifier transfers depends on *proof structure* (whether per-component reduction is useful). In the most favorable case—goal shape matches *and* per-component reduction is useful—the entire schema transfers and may even *improve* the existing proof.

## 7 Failures and Their Diagnoses

The 8 failures share a common meta-diagnosis: condensed mathematics was designed to *eliminate* the structures that probability-theory tactics operate on.

**Item 20:** `filter_upwards`**.** Requires `Filter.Eventually` goals. Condensed mathematics has zero uses of the `Filter` API across all 33 Mathlib files. The top-1 target (`isDiscrete_tfae`) proves equivalence of 7 characterizations of discreteness—all categorical (adjunction counit isomorphisms, essential image membership, colimit characterizations). This is a paradigm



*Citation: Linhares, A. "Yanasse: Finding New Proofs from Deep Vision's Analogies — Part 3," Argolab.org Report for Dissemination, 2026.*

incompatibility: condensed math was designed to replace filter-based topology with sheaf conditions.

**Item 23:** `all_goals fun_prop`**.** Requires multiple simultaneous `Measurable`/`Continuous` goals. The target (`isDiscrete_tfae`) is a serial `tfae_have` chain: never more than one goal open. Only 1 use of `fun_prop` exists in all of `Mathlib/Condensed/`.

**Item 24:** `lift`**.** Requires `CanLift` instances (e.g., $\mathbb{R}^{\infty}_{\geq 0} \to \mathbb{R}_{\geq 0}$). Condensed has zero `CanLift` instances, zero `WithTop`/`ENat`/`ENNReal` types. Replicates items 5 (RepTheory) and 14 (SetTheory).

**Item 25:** `any_goals`**.** Requires multi-goal batching structure. The target (`left_triangle_components`) has a single sequential goal at every proof step. Source dispatches 4 symmetric bilinear-form sub-goals; target has no algebraic branching. Contrast with RepTheory (item 6, success: genuine multi-goal) and SetTheory (item 15, success: arithmetic side-conditions).

**Item 26:** `measurability`**.** A $\sigma$-algebra closure tactic (`aesop` with `@[measurability]` rule set). Zero measurability goals exist in condensed proofs. Replicates items 7 (RepTheory) and 16 (SetTheory).

**Item 27:** `by_cases`**.** Requires conditional definitions (`ite`/`dite`) to unfold. Condensed has zero conditional definitions across all 33 files. The target (`counit` naturality) is definitional: `ext; rfl`. Contrast with SetTheory (item 17, success: infinite/finite dichotomy) and RepTheory (item 8, limited success: degenerate case split).

**Item 28:** `congrm`**.** Requires both sides of an equality to share a composite operator skeleton ($\int$, $\prod$, $\in$). Target goal is an adjunction triangle identity ($f \gg g = \nVdash$)—fundamentally asymmetric (composition vs. identity). Replicates items 9 (RepTheory) and 18 (SetTheory).

**Item 29:** `split_ifs`**.** Requires `ite`/`dite` subexpressions in the goal. Zero `if-then-else` constructs exist in any condensed definition or proof across all 33 files. Probability uses `split_ifs` for indicator functions, density processes, and truncations—all domain-locked to areas with conditional definitions.

**The common root cause.** Six of the eight failures (`filter_upwards`, `all_goals fun_prop`, `lift`, `measurability`, `by_cases`, `split_ifs`) fail because the tactic requires a *type-theoretic precondition* that condensed mathematics *by design* never satisfies:

| Schema | Required structure absent in Condensed |
|---|---|
| `filter_upwards` | `Filter`/`Filter.Eventually` |
| `all_goals fun_prop` | `Measurable`/`Continuous` goals |
| `lift` | `CanLift` instances |
| `measurability` | `@[measurability]`-tagged lemmas |
| `by_cases` | Conditional definitions (`dite`) |
| `split_ifs` | `ite`/`dite` subexpressions |

The remaining two (`any_goals`, `congrm`) fail for structural reasons: `any_goals` requires multi-goal proof states that condensed proofs' serial structure never produces, and `congrm` requires symmetric composite expressions that categorical morphism equalities lack.

# 8 Discussion

## 8.1 The 20% success rate

The low success rate is informative, not disappointing. It confirms a prediction: when the target area was *designed* to eliminate the source area's mathematical vocabulary, only schemas that operate at the level of proof *architecture* (decomposition into components) rather than proof *vocabulary* (filters, $\sigma$-algebras, conditional definitions) can transfer.

The 20% rate is consistent with the taxonomy from Part 1: of the 10 schemas, 6 are type-gated (expected to fail), 2 are structurally sensitive (`any_goals`, `congrm`—also expected to fail in this target), and only 2 (`congr with`, `ext1`) are pure decomposition patterns compatible with categorical equality goals.

## 8.2 Convergence to a single target theorem

Both successful transfers and 6 of the 8 failures target one of just two theorems: `topCatAdjunction.left_triangle_components` and `isDiscrete_tfae`. This convergence reflects the small target pool (189 entries): the QAP matcher has limited diversity to choose from, and a few high-scoring "attractor" entries dominate. A larger condensed-mathematics formalization would likely yield more diverse targets.

## 8.3 The `set_option` finding

The most practically valuable result is P1's discovery that `congr with x` can replace `ext` without the `set_option backward.isDefEq.respectTransparency false` workaround. This is not a theoretical observation: it produces a compilable, `sorry`-free proof that is strictly simpler than the Mathlib original.

The finding generalizes: any morphism equality in a concrete category (TopCat, CompHaus, Profinite) where `ext` requires the transparency override is a candidate for `congr with` replacement. The QAP-matched rank-2 analogue (`Light/TopCatAdjunction.lean`) has the same proof structure and likely benefits from the same replacement—a testable prediction.

## 8.4 Cost

The full pipeline for this pair consumed:

- $\sim$5 minutes of GPU time per schema (MPS) for analogy matching ($8 \times 189$ comparisons)—the fastest pair in the series.
- $\sim$6 hours of AI reasoning across 10 semantic adaptation sessions.
- $\sim$\$80–120 in API cost.

# 9 Cumulative Results After Three Pairs

Across 30 schema transfer attempts on 3 target areas, 10 schemas succeeded (33%), producing 12 Lean-verified new proofs. The "Proofs" column exceeds "Successes" for Set Theory because item 19 (`push`) yielded 3 independent proof variants. The number of distinct target theorems receiving new proofs is 5:

Table 3: Cumulative results across three Yanasse pairs.

| Pair | Schemas | Successes | Rate | Proofs | Pool |
|---|---|---|---|---|---|
| Prob → RepTheory (Part 1) | 10 | 4 | 40% | 4 | 672 |
| Prob → SetTheory (Part 2) | 10 | 4 | 40% | 6 | 2,076 |
| Prob → Condensed (Part 3) | 10 | 2 | 20% | 2 | 189 |
| **Total** | **30** | **10** | **33%** | **12** | — |

1. `Coinvariants.le_comap_ker` (RepTheory, 2 proofs)
2. $\varepsilon$`ToSingle`$_0$`_comp_eq` (RepTheory, 2 proofs)
3. `mul_eq_left_iff` (SetTheory, 3 schemas → 3 proofs)
4. `omega0_lt_preOmega_iff` (SetTheory, 1 schema → 3 variants)
5. `left_triangle_components` (Condensed, 2 proofs)

The most valuable individual result across all three papers is the `congr with x` proof of `left_triangle_components`, which is the only transfer that produces a *strictly better* proof than the Mathlib original.

## 10 Conclusion

The Probability → Condensed pair achieves a 20% transfer success rate, the lowest in the series, confirming that condensed mathematics' deliberate elimination of measure-theoretic vocabulary creates a hard barrier for most probability-theory schemas.

The positive results are correspondingly more valuable. The transferred `congr with x` tactic produces a proof that is *strictly better* than the Mathlib original—the only such result in the series. The finding that congruence-based decomposition can replace extensionality-based decomposition while avoiding transparency issues is a concrete, testable insight that probability theory's "go pointwise" convention contributes to condensed mathematics.

Cross-pair comparison across three target areas now confirms the type-gated / architecture dichotomy with statistical robustness: 0/9 transfers for type-gated schemas, 8/15 for architecture-sensitive schemas, with target-specific compatibility determining the outcome. The permanently failing schemas (`lift`, `measurability`, `congrm`) can be pruned from future non-measure-theoretic runs.

Part 4 will apply the method to `Probability` → `FieldTheory`, the current target of the running Ralph Loop.

## Acknowledgments

Named after Horacio Hideki Yanasse. The Deep Vision matching engine was originally developed for chess cognition research; its application to formal mathematics was enabled by the domain-independent design of `deep_vision_lib.py`. All computations were performed on a MacBook Air (Apple M-series, 32 GB unified memory) using PyTorch 2.8 on the MPS backend.



*Citation: Linhares, A. "Yanasse: Finding New Proofs from Deep Vision's Analogies — Part 3," Argolab.org Report for Dissemination, 2026.*

## References


[1] A. Linhares, "Yanasse: Finding new proofs from Deep Vision's analogies — Part 1: Probability Theory → Representation Theory," ARGO LABS Report for Distribution, April 2026.

[2] A. Linhares, "Yanasse: Finding new proofs from Deep Vision's analogies — Part 2: Probability Theory → Set Theory," ARGO LABS Report for Distribution, April 2026.

[3] A. Linhares, "Deep Vision," ARGO LABS Internal Report, 2026.

[4] D. R. Hofstadter and the Fluid Analogies Research Group, *Fluid Concepts & Creative Analogies: Computer Models of the Fundamental Mechanisms of Thought*. New York: Basic Books, 1995.

[5] D. R. Hofstadter, *Gödel, Escher, Bach: An Eternal Golden Braid*, 20th anniversary ed. New York: Basic Books, 1999.

[6] D. R. Hofstadter and E. Sander, *Surfaces and Essences: Analogy as the Fuel and Fire of Thinking*. New York: Basic Books, 2013.

[7] A. Linhares, "An active symbols theory of chess intuition," *Minds and Machines*, vol. 15, pp. 131–151, 2005.

[8] A. Linhares and P. Brum, "Understanding our understanding of strategic scenarios: What role do chunks play?" *Cognitive Science*, vol. 31, no. 6, pp. 989–1007, 2007.

[9] A. Linhares and A. E. T. A. Freitas, "Questioning Chase and Simon's (1973) 'Perception in Chess'," *New Ideas in Psychology*, vol. 28, no. 1, pp. 64–78, 2010.

[10] A. Linhares, A. E. T. A. Freitas, A. Mendes, and J. S. Silva, "Entanglement of perception and reasoning in the combinatorial game of chess," *Cognitive Systems Research*, vol. 13, no. 1, pp. 72–86, 2012.

[11] A. Linhares and D. M. Chada, "What is the nature of the mind's pattern-recognition process?" *New Ideas in Psychology*, vol. 31, no. 2, pp. 108–121, 2013.

[12] A. Linhares, "The emergence of choice: Decision-making and strategic thinking through analogies," *Information Sciences*, vol. 259, pp. 36–56, 2014.

[13] A. Linhares, "Deep Vision: Seeing the essence of proofs through analogy," ARGO LABS Internal Report, 2026.

[14] A. Linhares, "Deep Vision: A formal proof of Wolstenholme's theorem in Lean 4," ARGO LABS Report for Distribution, 2026.

[15] A. Linhares, "A sketch of Deep Vision's study of mathematics," ARGO LABS Report for Distribution, 2026.

[16] P. Scholze, "Lectures on condensed mathematics," Lecture notes, University of Bonn, 2019. Available at `https://www.math.uni-bonn.de/people/scholze/Condensed.pdf`.

[17] D. Clausen and P. Scholze, "Condensed mathematics and complex geometry," Lecture notes, 2022. Available at `https://people.mpim-bonn.mpg.de/scholze/Complex.pdf`.

[18] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Graduate Texts in Mathematics, vol. 5. New York: Springer-Verlag, 1998.

[19] D. Ásgeirsson, R. Brasca, N. Castellana, F. Filipazzi, E. Kudryashov, P. Lezama, and A. Topaz, "Categorical foundations of formalized condensed mathematics," *J. Symbolic Logic*, 2024. arXiv:2407.12840.

[20] D. Ásgeirsson, "A formal characterization of discrete condensed objects," 2024. arXiv:2410.17847.

[21] P. Scholze, "Liquid tensor experiment," Blog post on Xena Project, December 2020.

[22] J. Commelin, A. Topaz, et al., "Completion of the Liquid Tensor Experiment," Lean Community Blog, 2022.

# A Lean Code: New Proofs

## A.1 Item 21: `left_triangle_components` via `congr with x` (no `set_option`)

```
import Mathlib.Condensed.TopCatAdjunction
import Mathlib.Topology.Category.CompactlyGenerated

open Condensed CondensedSet CategoryTheory CompHaus

universe u

-- Transferred proof: congr with x (from Probability)
-- NOTE: No set_option needed!
example (Y : CondensedSet.{u}) :
    condensedSetToTopCat.map
      (topCatAdjunctionUnit Y) ≫
      topCatAdjunctionCounit
        (condensedSetToTopCat.obj Y) =
          𝟙 _ := by
  congr with x
  change Y.obj.map (𝟙 _) x = x
  simp
```



*Citation: Linhares, A. "Yanasse: Finding New Proofs from Deep Vision's Analogies — Part 3," Argolab.org Report for Dissemination, 2026.*

### A.2 Item 22: `left_triangle_components` via `ext1 x` (three variants)

```
import Mathlib

open CategoryTheory Condensed CondensedSet CompHaus

universe u

-- Variant A: ext1 x + change + simp
set_option
    backward.isDefEq.respectTransparency false in
theorem left_triangle_A (Y : CondensedSet.{u}) :
    condensedSetToTopCat.map
      (topCatAdjunctionUnit Y) ≫
      topCatAdjunctionCounit Y.toTopCat =
        𝟙 _ := by
  ext1 x
  change Y.obj.map (𝟙 _) x = _
  simp

-- Variant B: ext1 x + change + simp only
set_option
    backward.isDefEq.respectTransparency false in
theorem left_triangle_B (Y : CondensedSet.{u}) :
    condensedSetToTopCat.map
      (topCatAdjunctionUnit Y) ≫
      topCatAdjunctionCounit Y.toTopCat =
        𝟙 _ := by
  ext1 x
  change Y.obj.map (𝟙 _) x = _
  simp only [FunctorToTypes.map_id_apply,
    condensedSetToTopCat_obj_carrier,
    TopCat.hom_id, ContinuousMap.id_apply]

-- Variant C: ext1 x + change + rw + rfl
set_option
    backward.isDefEq.respectTransparency false in
theorem left_triangle_C (Y : CondensedSet.{u}) :
    condensedSetToTopCat.map
      (topCatAdjunctionUnit Y) ≫
      topCatAdjunctionCounit Y.toTopCat =
        𝟙 _ := by
  ext1 x
  change Y.obj.map (𝟙 _) x = _
  rw [FunctorToTypes.map_id_apply]
  rfl
```



*Citation: Linhares, A. "Yanasse: Finding New Proofs from Deep Vision's Analogies — Part 3," Argolab.org Report for Dissemination, 2026.*